

Bitácora de desarrollo del Frontend

Interfaz de visualización del grafo de Productos, Insumos y Proveedores

Información general

Proyecto: Interfaz web para visualización del grafo de Productos, Insumos y Proveedores

Tecnología: React (Vite) + Cytoscape.js, consumiendo la API REST en FastAPI

Despliegue: Netlify

Asistente de IA: Claude Code (Anthropic)

Esta bitácora documenta, a modo de registro, los prompts e interacciones utilizados durante el desarrollo del frontend, complementando la bitácora del backend. El frontend consume los endpoints `/productos`, `/insumos` y `/proveedores` de la API para renderizar el grafo de forma interactiva.

Registro de prompts

Prompt 1: Visualización de un nodo

Herramienta: Claude Code

Prompt (versión redactada):

Necesito representar visualmente cada nodo del grafo (producto, insumo o proveedor) en el canvas de la aplicación, diferenciando claramente la capa a la que pertenece mediante color y forma. Cada nodo debe mostrar su nombre como etiqueta y debe resaltarse visualmente al pasar el mouse por encima o al seleccionarlo.

Objetivo: Definir la representación visual base de los nodos, de forma que el usuario pueda identificar el tipo de entidad de un vistazo, sin depender únicamente del texto.

Requerimientos de diseño solicitados:

- **Librería de grafos:** se evaluaron `vis-network` y `Cytoscape.js`; se eligió `Cytoscape.js` por su control fino sobre estilos.
- **Diferenciación por capa:** color y forma distintos para producto, insumo y proveedor.
- **Etiquetas:** nombre del nodo visible sobre el elemento.
- **Interacción:** estado visual de hover y de selección (resaltado/opacidad).

Resultado obtenido: Claude Code generó un componente `GraphCanvas.jsx` que inicializa `Cytoscape.js` con una hoja de estilos (stylesheet) basada en la propiedad `capa` de cada nodo: azul para producto, verde para insumo y naranja para proveedor, con formas diferenciadas (rectángulo redondeado, elipse y diamante respectivamente). Se agregaron reglas `:selected` y `hover` para resaltar el nodo activo.

Prompt 2: Conexión del nodo con arista

Herramienta: Claude Code

Prompt (versión redactada):

Las aristas deben representar la relación REQUIERE entre nodos, con una flecha dirigida desde el nodo de origen hacia el nodo de destino. Cuando el grafo crece y un nodo tiene varias conexiones, necesito que al seleccionar ese nodo se resalten únicamente sus aristas y nodos directamente conectados, y que el resto se atenúe.

Objetivo: Representar correctamente la direccionalidad de las dependencias y facilitar la lectura de las relaciones a medida que el grafo aumenta de tamaño.

Requerimientos de diseño solicitados:

- **Aristas dirigidas:** marcador de flecha hacia el nodo destino.
- **Curvado de aristas:** estilo bezier para evitar solapamientos entre conexiones paralelas.
- **Resaltado de vecinos:** al hacer clic en un nodo, resaltar solo sus vecinos directos (origen y destino).
- **Atenuación:** reducir la opacidad de los nodos y aristas no relacionados con la selección.

Resultado obtenido: Se definió el estilo de arista con `target-arrow-shape triangle` y `curve-style bezier`. Se implementó una función `resaltarVecinos(nodoId)` que usa los métodos `connectedEdges()` y `neighborhood()` de Cytoscape para aplicar una clase CSS "activo" a los elementos relacionados y una clase "atenuado" al resto del grafo.

Prompt 3: Contexto de los nodos

Herramienta: Claude Code

Prompt (versión redactada):

Al hacer clic sobre un nodo quiero que se abra un panel lateral con su información: id, nombre, capa, categoría y la lista de nodos con los que se relaciona directamente (de qué depende y quién depende de él). Esta información debe obtenerse a partir de los datos que ya entrega la API del backend, sin duplicar lógica de negocio en el frontend.

Objetivo: Brindar contexto detallado de cada entidad sin saturar visualmente el grafo, reutilizando la información que ya expone la API.

Requerimientos de diseño solicitados:

- **Panel lateral:** sidebar que se abre y cierra según la selección del nodo.
- **Metadatos:** mostrar id, nombre, capa y categoría del nodo.
- **Relaciones:** listas separadas en "Requiere a" y "Es requerido por".
- **Estados:** manejo de carga y de error al consultar la API.

Resultado obtenido: Se creó el componente `NodeDetailPanel.jsx` junto con un hook `useNodeContext(nodoid)` que consulta los endpoints existentes de la API y, en el cliente, filtra la lista de aristas para construir las relaciones entrantes y salientes del nodo seleccionado. El panel se muestra como un overlay lateral con una opción para cerrarlo y volver a la vista general del grafo.

Prompt 4: Cómo publicarlo

Herramienta: Claude Code

Prompt (versión redactada):

Ya tengo el frontend funcionando en local con Vite. ¿Cómo lo publico en Netlify conectado al repositorio de GitHub, y cómo configuro la variable de entorno que apunta a la URL de la API en producción para que no quede apuntando a localhost?

Objetivo: Dejar la aplicación accesible públicamente mediante un despliegue continuo, separando la configuración de entorno local de la de producción.

Requerimientos de diseño solicitados:

- **Despliegue continuo:** conectar el repositorio de GitHub con Netlify.
- **Build:** comando `npm run build` y carpeta de publicación `dist`.
- **Variable de entorno:** `VITE_API_URL` con la URL pública de la API.
- **Redirecciones:** reglas para que las rutas de React funcionen al recargar la página.

Resultado obtenido: Claude Code guió la configuración del archivo `netlify.toml` con el comando de build, la carpeta `dist` como publish directory y una regla de redirección (`/* -> /index.html`) para el enrutamiento del lado del cliente. También se documentó cómo declarar `VITE_API_URL` en las variables de entorno del panel de Netlify, de forma que el cliente HTTP del frontend la lea mediante `import.meta.env.VITE_API_URL` en lugar de una URL fija apuntando a localhost.

Explicación del código

GraphCanvas.jsx: el lienzo del grafo

Es el componente central de la aplicación. Inicializa Cytoscape.js con los nodos y aristas que llegan desde la API, aplica la hoja de estilos por capa y expone los eventos de clic y hover que disparan el resaltado de vecinos y la apertura del panel de contexto.

NodeDetailPanel.jsx: el contexto del nodo

Panel lateral que se activa cuando el usuario selecciona un nodo en el canvas. Muestra sus datos básicos y las relaciones calculadas a partir de las aristas ("Requiere a" / "Es requerido por"), dando una vista de detalle sin necesidad de saturar el grafo con etiquetas adicionales.

api.js: la conexión con el backend

Centraliza las llamadas HTTP hacia los endpoints /productos, /insumos y /proveedores de la API en FastAPI. Usa la variable de entorno VITE_API_URL para que la misma base de código apunte a localhost en desarrollo y a la URL pública de la API en producción, sin cambiar una sola línea de código entre ambos entornos.

netlify.toml: la publicación

Archivo de configuración leído por Netlify en cada despliegue. Define el comando de construcción, la carpeta que se publica y la regla de redirección necesaria para que la aplicación de una sola página (SPA) funcione correctamente al recargar o compartir enlaces directos a una vista específica.